

Highlights

Information Propagation Algorithm For Reliability Analysis In Acyclic Process Networks

Temitope Ohiani, Edoardo Patelli, Caroline Pyke

- Introduces an exact reliability computation for directed acyclic process networks with complex dependency structures
- Proposes a progressive diamond decomposition through conditional enumeration on shared fork ancestors
- Reduces computational complexity from $O(2^{\sum c_i} \times \prod d_i)$ to $O(\sum 2^{c_i})$ through super node replacement
- Demonstrates 134-times faster execution than existing message-passing methods while maintaining exact results
- Applied to a realistic network 782 node drone delivery network

Information Propagation Algorithm For Reliability Analysis In Acyclic Process Networks

Temitope Ohiani^a, Edoardo Patelli^a, Caroline Pyke^b

^a*Department of Civil and Environmental Engineering, University of Strathclyde, Glasgow, UK*

^b*United Kingdom National Nuclear Laboratory, UK*

Abstract

Signal reliability analysis in process systems requires computing the probability that viable paths exist from source nodes to the target. Traditional methods struggle with reconvergent path structures where multiple paths from common sources reconverge at downstream nodes, creating statistical dependencies that violate independence assumptions underlying standard reliability calculations.

This paper presents the Information Propagation Algorithm (IPA), an exact message-passing approach that resolves these dependencies through conditional enumeration on shared fork ancestors. IPA processes networks in topological layers, applying inclusion-exclusion for independent paths and progressive diamond decomposition for dependent structures. Diamond sub-graphs are systematically identified, solved through conditional probability enumeration, and replaced with statistically equivalent supernodes, reducing computational complexity from $O(2^{\sum c_i} \times \prod d_i)$ to $O(\sum 2^{c_i})$.

Two case studies are presented to demonstrate the computational advantages. On a 16-node grid network, IPA achieves 134-times faster execution than dPrPm while maintaining exact results. Analysis of a 782-node medical delivery network with 2,305 diamond dependencies shows 36

Keywords: Network Reliability, Directed Acyclic Graphs, Message Passing, Diamond Structures, Process Systems, Infrastructure Networks, Belief Propagation, Inclusion-Exclusion

Email addresses: `temitope.ohiani@strath.ac.uk` (Caroline Pyke), `edoardo.patelli@strath.ac.uk` (Caroline Pyke), `caroline.pyke@uknnl.com` (Caroline Pyke)

1. Introduction

Resource and signal propagation form the backbone of many modern civil infrastructure systems, including power grids, water distribution networks, and telecommunication systems. These systems, which we refer to in this work as multi-source process networks, refer to resource flow systems where input, such as materials, information, or signals, flows from source nodes through a series of paths to achieve a specific goal of transformation and/or distribution. Failures and breakdowns in these systems can have catastrophic consequences, ranging from economic and service losses to environmental and public threats. Hence, performance assessment and health monitoring of these types of systems focus on measuring the system’s ability to retain sufficient connectivity and maintain resource flow in the network.

Directed acyclic graphs (DAGs) are a natural fit to model and evaluate the performance of resource flow systems. The graph nodes represent system components or processing stages, while directed edges model permissible flow transition and dependencies between components. Consider the typical process of transport networks where passengers’ journeys depend on several related sequential processes, including the train lines themselves, where failures at earlier stations can cause disruption and delays in travel and, in some cases, stop service altogether.

To mitigate cascading failure scenarios and service bottlenecks, the reliability of the network’s resource flow is an important performance metric to monitor. Reliability is generally defined as the probability that a system performs its intended function over a specified time. In flow networks, the intended function is that the other parts of the network successfully receive information flowing from the sources, and complex systems have redundancy strategies and backup paths to ensure this flow. Reliability in multi-source flow networks is related to the probability that at least one viable operational path exists from source nodes to the other nodes in the network.

The classic approach to exactly solving the reliability problem involves enumerating and combining all possible source-to-node paths. However, this quickly becomes computationally intractable for complex process systems with multiple sources, extensive redundancy, and re-convergent network topologies [1]. The structure of process systems aligns naturally with message-passing algorithms, where each node’s reliability can be viewed as

its ability to receive messages, i.e. signals from upstream sources. Each node's reliability can be viewed as its ability to receive signals from upstream sources. Rather than enumerating exponentially many paths globally, reliability can instead be implemented as a distributed computation where each node locally calculates: "What is the probability I receive at least one viable signal from any source?" based only on messages from its immediate neighbours.

To this end, this work presents a message passing approach that computes exact signal reliability for DAG networks by progressively decomposing re-convergent path structures (diamonds) through conditional enumeration on shared fork ancestors named Information Propagation Algorithm (IPA). Being based on analytical methods, the algorithm allows to analyse large systems in nearly real-time making it adaptable during design phases where multiple analysis of the system under different configurations need to be tested as well for supporting decision tools of critical systems that required a fast assessment of the systems.

2. Related Works

Early reliability analysis methods are based on relatively straightforward mathematical principles that easily apply to simpler process systems. The Reliability Block Diagram (RBD)[2, 3, 4] method models the system as connected component blocks whose reliability can be intuitively calculated based on their arrangement. For simple systems, RBDs can be very fast and accurate. However, the system representation is limited to a mix of the different configuration types (series, parallel, and k-out-of-n). The Dynamic Reliability Block Diagrams (DRBD) [5] extends traditional RBD by allowing for more realistic modelling of component interactions and system dependencies. However, DRBD models still require significant overhead for systems with highly interconnected structures and struggle with networks with multiple convergent paths.

Boolean logic-based methods compute network reliability by modelling the system as a comprehensive view of event causes and consequences. Fault Tree Analysis (FTA) [6] is a top-down deductive logic tree to identify possible combinations of component failures that could cause specific undesirable events. Event Tree Analysis (ETA) is the complementary inductive method where the logic tree instead begins with the initiating event and systematically traces all possible sequences of subsequent successes and failures

through branching structures and logic gates [7].

Both FTA and ETA require extensive manual construction and maintenance for larger and more complex processes without further extensions. Although more flexible than RBDs, both tree analysis methods struggle and cannot efficiently handle complicated configurations or adaptive system behaviours common in large-scale process networks [8, 6].

The main limitations of tree-based methods stem from exponential growth in logic expressions and manual construction requirements for complex systems. Binary Decision Diagrams (BDDs)[9, 10, 11] were adapted from circuit design to improve fault tree models by using variable elimination and path reduction techniques on Boolean functions. In BDDs, Boolean functions are represented as directed acyclic graphs (DAGs) based on Shannon decomposition, where each non-sink node represents a Boolean variable with two outgoing edges corresponding to the variable being true or false [6]. This structure allows BDD-based methods to require less memory and computational time than traditional fault tree analysis [9]. BDD algorithms can handle complex Boolean expressions with thousands of variables while maintaining polynomial space complexity for many systems. However, this critically depends on variable ordering, with poor ordering choices leading to exponential size growth[12]. For highly interconnected networks with shared components and reconvergent fanout structures, optimal variable ordering becomes computationally intractable. BDD construction for networks with diamond structures - where multiple independent paths converge at common nodes - suffers from the reconvergence problem, requiring extensive intermediate node generation that can exceed memory limitations even for moderately sized systems. These computational challenges become particularly severe when modelling common cause failures and complex dependency structures that propagate through multiple interconnected pathways simultaneously.

Common cause failures create statistical dependencies that violate the independence assumptions underlying traditional reliability calculations[13]. When multiple components share common failure modes, the resulting dependency patterns cannot be efficiently captured through simple Boolean combinations, representing a fundamental limitation shared by RBDs, FTAs, and BDDs in process systems with convergent network topologies.

To address the dependency modelling limitations of classical reliability methods, several network path-based approaches have emerged that better handle common cause failures and complex dependency structures in interconnected systems. These methods contextualise the network as a proba-

bilistic graph theory problem and leverage probability mathematics and set operations to calculate network reliability. Analytical path-based methods focus on network connectivity; thus, the challenge lies in efficiently enumerating the source-sink paths. Simulation methods such as Monte Carlo (MC) have become a widely adopted reliability analysis method for complex networks [14]. MC algorithms can handle arbitrary network structures and provide confidence intervals, although they can become computationally expensive for large networks requiring many samples to reach acceptable accuracy. Monte Carlo variants such as importance sampling[15] and subset simulation [16] have been proposed to address rare failure events, though these optimisations often demand more computational costs. To evaluate path connectivity between source and sink nodes, recursive decomposition methods take the approach of breaking down the network graph into smaller subproblems. Minimal path and cut enumeration approaches [17, 18, 2, 19] identify the smallest collections of components that ensure connectivity or guarantee system failure. The cut-based method in [19] proposes simultaneously using both the disjoint minimal cut sets and disjoint minimal path sets during the decomposition process for networks with low edge reliabilities. Kim and Kang’s method [20] extends the Recursive Decomposition Algorithm through a Logical Expansion approach (LE-RDA) that uses two separate procedures for intersection and union logical functions to handle decomposition of general system events with multiple node pairs.

On the other hand, message passing methods [21, 22] aim to infer the reliability of the system by exploiting the network structure to propagate the local calculations throughout the network until convergence. Based on belief propagation theory [23], these approaches treat reliability as a distributed inference problem on graphical models, where probability messages are iteratively passed between network nodes. Unlike enumeration-based methods, message passing scales linearly with network size and naturally handles complex dependency structures. However, convergence is always guaranteed [24], and approximation quality depends heavily on network topology and component correlation strength.

3. Network Model

A process system is modelled as a directed acyclic graph $G = (V, E)$ where V denotes the set of nodes (components) and $E \subseteq V \times V$ represents

the set of directed edges (dependencies). Let $S \subseteq V$ and $T \subseteq V$ denote the sets of source and sink nodes, respectively.

Definition 3.1 (Viable Path). *A viable path $\pi(s, v) = (s = v_0, v_1, \dots, v_k = v)$ from source $s \in S$ to node $v \in V$ exists if all nodes and edges along the path are functional.*

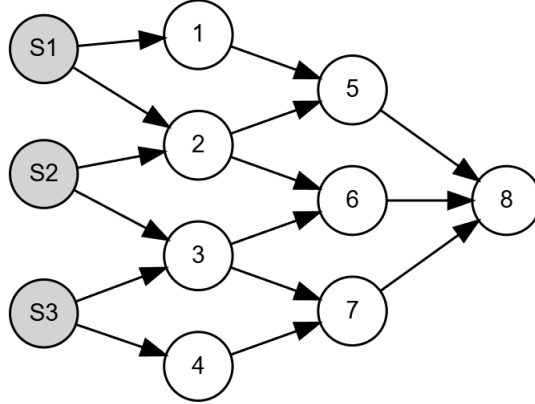


Figure 1: A Multi-Source Directed Acyclic Graph.

The reliability problem is formally stated as computing the probability that there is at least one viable path from any source node to each target node in the network.

3.1. Mathematical Notation

- $R(v) \in [0, 1]$: intrinsic reliability (prior probability) of node $v \in V$
- $P(u, v) \in [0, 1]$: transmission probability of edge $(u, v) \in E$
- $\text{Pa}(v)$: set of parent nodes of v
- $\text{Ch}(v)$: set of child nodes of v
- $b(v)$: belief (computed reliability) at node v
- $\pi(s, v)$: directed path from source $s \in S$ to node $v \in V$
- $\Pi(s, v)$: set of all directed paths from source s to node v

3.2. Network Assumptions

Assumption 3.1 (Network Properties). *The network G satisfies: (i) acyclicity - G contains no directed cycles, and (ii) connectivity - every non-source node is reachable from at least one source.*

Assumption 3.2 (Independence Properties). *(i) Node states are statistically independent given their parents, (ii) edge states are statistically independent, and (iii) each node and edge has binary states: functional or failed.*

4. The Information Propagation Algorithm

The Information Propagation Algorithm (IPA) computes reliability through signal reception probability. For any node $v \in V$, the main belief update equation is:

$$b(v) = R(v) \times P(v \text{ receives at least one signal from sources}) \quad (1)$$

Consider node v with parents $\text{Pa}(v) = \{p_1, p_2, \dots, p_k\}$. Each parent p_i transmits a signal with probability:

$$S_i = b(p_i) \times P(p_i, v) \quad (2)$$

The goal is to compute $P(v \text{ receives at least one signal})$, which corresponds to $P(A_1 \cup A_2 \cup \dots \cup A_k)$ where each event A_i represents "signal reaches v via parent p_i ".

Like other belief propagation methods discussed in Section 2, IPA computes signal reliability in DAGs through an iterative message passing process where each node updates its belief based on messages received from its parents.

4.1. Dependency-First Belief Propagation

Given a set of nodes in the input DAG, the information propagation algorithm updates nodes in a "dependency-first" priority order based on the topological sorting property of DAG networks. This node processing sequence leverages the hierarchical structure of directed graphs to propagate and update information from source to sink nodes.

For the directed acyclic graph $G = (V, E)$, the topological sorting T is a linear ordering of nodes where every node appears exactly once, and for every edge $u \rightarrow v$, node u must appear before v . The algorithm constructs

this ordering by iteratively identifying source nodes (nodes with no incoming edges), removing them and their outgoing edges, then repeating until all nodes are processed into ordered layers $\{L_1, L_2, \dots, L_k\}$.

Theorem 4.1 (Convergence Guarantee). *Given a DAG $G = (V, E)$, the belief propagation algorithm converges in exactly $|L|$ iterations, where L represents the topologically sorted layers of G .*

Proof. Let $L = \{L_1, \dots, L_k\}$ be the topologically sorted layers of G . For any node $v \in L_i$, all its parents $p \in \text{Pa}(v)$ must be in layers L_j where $j < i$. Therefore, when processing layer L_i , all parent beliefs are final. Each node is processed exactly once, and its belief value is never revisited. Thus, convergence is guaranteed in $|L|$ iterations. $\square$

This topologically sorted processing sequence provides three key properties: all parent beliefs are computed before children, eliminating the need for recursive backtracking; each node's belief is computed exactly once from all of its parents' messages, preventing redundant calculations; and parallel processing is possible within layers.

Given the processing layers, IPA's belief propagation sequence begins by initialising source nodes with their prior probabilities, then processes each subsequent layer by applying belief update rules based on the signals received from parent nodes. Within each processing layer, the belief of each node v is simultaneously computed based on the belief updating rules discussed in the next sections.

5. Belief Updates for Independent Path Structures

The node belief update rules define how messages from parents are propagated to nodes. The reliability belief at a node depends on the dependency structure of the paths from its parents. When these paths are independent, standard probability rules for independent events apply directly to compute the probability of signal reception.

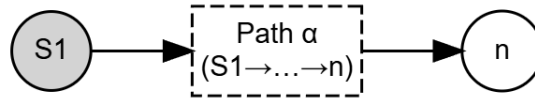


Figure 2: Single path structure: chained signal propagation

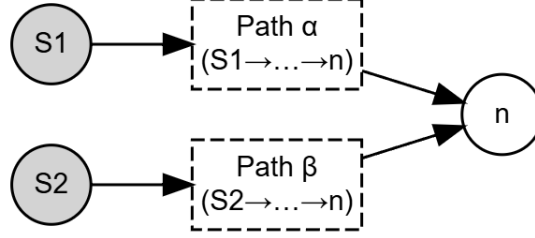


Figure 3: Multiple independent path structure: parallel signal propagation

Consider a node with n independent paths, where A_i denotes the event that a signal successfully reaches the node via path i . The probability that at least one signal reaches the node is given by the inclusion-exclusion formula:

$$P\left(\bigcup_{i=1}^n A_i\right) = \sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \dots < i_k \leq n} P\left(\bigcap_{j=1}^k A_{i_j}\right) \quad (3)$$

Under the independence assumption, each intersection term becomes:

$$P(A_{i_1} \cap \dots \cap A_{i_k}) = R_{i_1} \cdot R_{i_2} \cdot \dots \cdot R_{i_k} \quad (4)$$

In chained paths, success requires all components in the path to work, corresponding to the intersection $P(A_1 \cap A_2 \cap \dots \cap A_n)$:

$$R_{\text{chained}} = \prod_{i=1}^n R_i \quad (5)$$

For multiple independent paths, the system succeeds if any path works, corresponding to $P(A_1 \cup A_2 \cup \dots \cup A_n)$:

$$R_{\text{multiple paths}} = 1 - \prod_{i=1}^n (1 - R_i) \quad (6)$$

In a DAG $G = (V, E)$, the belief $b(v)$ at node v represents the reliability, which is the probability of a viable path from source nodes to v . The number of parents a node depends on determines how its reliability is calculated. The belief update procedure depends on node type:

The belief at any source node is taken as the node's prior probability:

$$b(v_s) = R(v_s) \quad (7)$$

Any node with a single parent (Figure 2) is processed similarly to a chained path system where the node's prior probability, the parent's belief, and the transmission probability of the edge between them are all connected in series:

$$b(v) = R(v) \times b(p) \times P(p, v) \quad (8)$$

Any node with multiple unrelated parents (Figure 3) is processed like a system where each edge from each parent operates independently, and these combinations are then in parallel with each other. For independent signals $\{S_1, S_2, \dots, S_k\}$ from parents:

$$P(\text{at least one signal}) = \sum_{i=1}^k (-1)^{i+1} \sum_{|I|=i} \prod_{j \in I} S_j \quad (9)$$

This expands to:

$$P(\text{at least one signal}) = \sum S_i - \sum_{i < j} S_i S_j + \sum_{i < j < k} S_i S_j S_k - \dots \quad (10)$$

The inclusion-exclusion approach accounts for overlapping events by adding individual probabilities, subtracting pairwise intersections, adding triple intersections, and continuing this alternating pattern until all overlaps are counted exactly once. The final belief becomes:

$$b(v) = R(v) \times \left[1 - \prod_{p \in \text{Pa}(v)} (1 - b(p) \times P(p, v)) \right] \quad (11)$$

This equation is mathematically equivalent to the Noisy-OR equation in probabilistic graph models, where each parent can independently cause the child to be active with some specified probability, and the child node only fails if all of its parents fail.

6. Diamond Structures and the Independence Problem

Definition 6.1 (Diamond Structure). *A diamond structure is a sub-graph $D \subseteq G$ consisting of:*

- *A set of fork nodes $F = \{f_1, f_2, \dots, f_k\}$ where each f_i has $|Ch(f_i)| \geq 2$*

- A single join node j with $|Pa(j)| \geq 2$
- A set of at least two distinct directed paths that diverge from nodes in F and reconverge at j
- All intermediate nodes and edges between the fork-join pairs are contained within D

Although the Noisy-OR equation correctly handles nodes with independent parent paths, diamond structures require special consideration because of the statistical dependencies they introduce. When multiple paths from the same fork node reconverge at a join node, the success or failure of these paths is no longer independent since they share a common cause. These shared dependency patterns create diamond structures that break the independence assumption underpinning belief updates in non-diamond nodes.

6.1. Mathematical Example: Simple Diamond

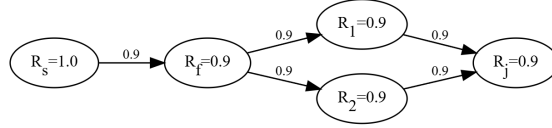


Figure 4: A basic diamond sub-graph between fork node f and join node j

The simplest type of diamond (an induced diamond) is made up of exactly two paths between a single fork-join pair, with all affected nodes and edges fully contained within the subgraph. Consider Figure 4, where all non-source nodes have an intrinsic reliability of 0.9 and all edges have a transmission probability of 0.9.

6.1.1. Solution by Path Enumeration (Correct but Expensive):

We enumerate all possible states of the fork and intermediate nodes:

$$\begin{aligned}
 P_1 &= 0.81 \times 0.81 \times 0.81 = 0.531441 && \text{(Fork active, both intermediates work)} \\
 P_2 &= 0.81 \times 0.81 \times 0.19 = 0.124659 && \text{(Fork active, only first intermediate works)} \\
 P_3 &= 0.81 \times 0.19 \times 0.81 = 0.124659 && \text{(Fork active, only second intermediate works)} \\
 P_4 &= 0.81 \times 0.19 \times 0.19 = 0.029241 && \text{(Fork active, both intermediates fail)} \\
 P_5 &= 0.19 && \text{(Fork inactive, no signal possible)}
 \end{aligned}$$

Signal transmission probabilities are:

$$P(\text{signal}) = \begin{cases} 0.99 & \text{if both paths send signals (State 1)} \\ 0.9 & \text{if only one path sends a signal (States 2,3)} \\ 0 & \text{if no signals (States 4,5).} \end{cases}$$

Thus, the total probability of signal propagation via diamond is

$$\begin{aligned} P(\text{signal}) &= P_1 \times 0.99 + P_2 \times 0.9 + P_3 \times 0.9 \\ &= 0.531441 \times 0.99 + 0.124659 \times 0.9 + 0.124659 \times 0.9 \\ &= 0.75051279. \end{aligned}$$

The final belief at node j is

$$b(j) = R(j) \times P(\text{signal}) = 0.9 \times 0.75051279 = 0.675461511.$$

6.1.2. Solution by Noisy-OR (Incorrect):

Beliefs at intermediate nodes (topological order):

$$\begin{aligned} b(\text{fork}) &= R_s \times P(\text{edge } s \rightarrow f) \times R_f = 1.0 \times 0.9 \times 0.9 = 0.81, \\ b(p_1) = b(p_2) &= b(\text{fork}) \times P(\text{edge } f \rightarrow p) \times R_p = 0.81 \times 0.9 \times 0.9 = 0.6561. \end{aligned}$$

Signal from each intermediate to j :

$$S_1 = S_2 = b(p_i) \times P(\text{edge } p \rightarrow j) = 0.6561 \times 0.9 = 0.59049.$$

Treating paths as independent:

$$\begin{aligned} P(\text{signal}) &= 1 - (1 - 0.59049)^2 = 0.83227, \\ b(j) &= R(j) \times P(\text{signal}) = 0.9 \times 0.83227 = 0.749043. \end{aligned}$$

6.1.3. Solution by Diamond Conditioning (Correct and Efficient):

Fork belief:

$$b(\text{fork}) = R_s \times P(\text{edge } s \rightarrow f) \times R_f = 1.0 \times 0.9 \times 0.9 = 0.81.$$

Conditional probabilities:

$$\begin{aligned} \text{If fork} = 1: \quad \text{Path prob.} &= P(\text{edge } f \rightarrow p) \times R_p \times P(\text{edge } p \rightarrow j) = 0.9 \times 0.9 \times 0.9 = 0.729, \\ &P(\text{signal} \mid \text{fork} = 1) = 1 - (1 - 0.729)^2 = 0.926559. \\ \text{If fork} = 0: \quad &P(\text{signal} \mid \text{fork} = 0) = 0. \end{aligned}$$

Total signal probability:

$$\begin{aligned}
P(\text{signal}) &= P(\text{signal} \mid \text{fork} = 1) \times P(\text{fork} = 1) + P(\text{signal} \mid \text{fork} = 0) \times P(\text{fork} = 0) \\
&= 0.926559 \times 0.81 + 0 \times 0.19 \\
&= 0.75051279.
\end{aligned}$$

Final belief:

$$b(j) = R(j) \times P(\text{signal}) = 0.9 \times 0.75051279 = 0.675461511.$$

The path enumeration method, while mathematically correct, becomes computationally expensive as the number of distinct parent paths increases. For this simple diamond with only two distinct paths, full enumeration still requires computing 5 separate network states. On the other hand, IPA's conditioning approach works by recognising that when the fork node state is deterministic, the diamond paths become independent again. This restored independence allows standard probability combination rules to apply within each conditioning state. By conditioning only on the shared dependency, the relative computation cost for the same diamond is significantly reduced. This advantage becomes more evident as diamond structures grow in size and complexity. For a diamond with n distinct paths from a single fork node, path enumeration scales to $O(2^n + 1)$ network states, while conditioning maintains constant complexity of 2 states regardless of the number of paths originating from that fork.

The algorithm's conditioning approach to resolving diamonds can be formalised for any single-fork diamond structure as follows:

Taking the complete sample space Ω of possible events at join node n_j , let F be the event that fork node n_f succeeds, and J be the event that n_j receives at least one signal through the diamond. The sample space is partitioned into disjoint events F and $\bar{F}$. By the law of total probability:

$$P(\text{signal through diamond}) = P(J|F)P(F) + P(J|\bar{F})P(\bar{F}) \quad (12)$$

where $P(F) = b(n_f)$ and $P(\bar{F}) = 1 - b(n_f)$. When the fork is active (F occurs), the diamond paths become independent, allowing standard inclusion-exclusion:

$$P(J|F) = 1 - \prod_{i=1}^k (1 - S_i) \quad (13)$$

where S_i is the probability of signal transmission through the i -th diamond path when the fork is active. The term $P(J|\bar{F})$ captures alternative signal paths to n_j when the fork fails. In simple induced diamonds, there are no alternative paths to the join node from other sources; therefore, $P(J|\bar{F}) = 0$. For complex diamond structures embedded within larger networks, $P(J|\bar{F})$ accounts for non-diamond parent signals and interconnected diamond influences. For diamond structures with multiple conditioning nodes $\mathcal{C} = \{c_1, c_2, \dots, c_m\}$, the algorithm considers all possible conditioning states:

$$b(n_j) = R(n_j) \times \sum_{s=0}^{2^m-1} P(\text{state}_s) \times \phi(\text{state}_s, n_j) \quad (14)$$

where:

$$P(\text{state}_s) = \prod_{i=1}^m [b(c_i)]^{b_i(s)} \times [1 - b(c_i)]^{1-b_i(s)} \quad (15)$$

Here $b_i(s)$ is the i -th bit of binary state index s , and $\phi(\text{state}_s, n_j)$ is the signal through the diamond given the conditioning state.

Once the signals from diamond paths have been resolved, the combined signal at the diamond's join nodes can then be computed as:

$$b(n_j) = R(n_j) \times \text{PIE}(\text{DiamondSignals} \cup \text{RegularSignals}) \quad (16)$$

7. Decomposing Diamond Structures

Real-world process systems rarely contain simple, isolated diamonds. In DAGs, diamond paths often overlap and nest within each other, creating cascading dependencies that grow exponentially with network size. As the algorithm goes deeper into these networks, the number of possible path combinations increases exponentially with both the number of shared fork nodes and the depth of nesting.

Without decomposition, resolving these types of subgraphs would involve simultaneously conditioning on the combination of states of every shared fork node. In diamonds with long paths and multiple intermediate convergence points, this quickly becomes intractable. For nested diamond structures, this requires multi-level conditional enumeration:

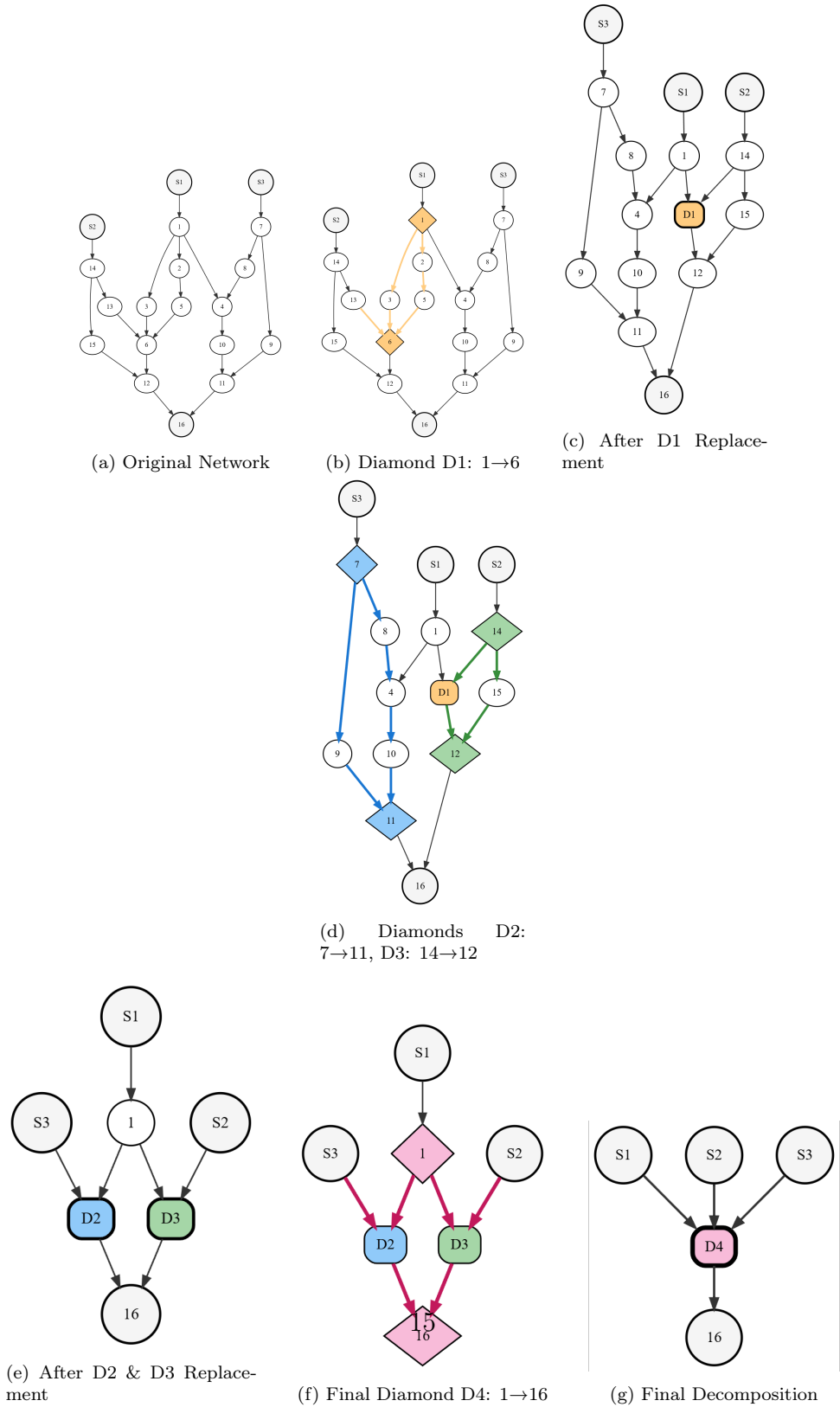


Figure 5: Progressive diamond decomposition showing how nested diamond structures are reduced to super nodes during topological processing.

$$b(n_j) = R(n_j) \times \sum_{s_1=0}^{2^{m_1}-1} \sum_{s_2=0}^{2^{m_2}-1} \cdots \sum_{s_L=0}^{2^{m_L}-1} P(\text{state}_{s_1}) \times P(\text{state}_{s_2}|\text{state}_{s_1}) \\ \times \cdots \times \phi_L(\text{state}_{s_1} \dots \text{state}_{s_L}) \quad (17)$$

where L represents the number of nesting levels, m_i is the number of conditioning nodes at level i , and the computation scales exponentially with both the depth and breadth of nested structures.

IPA's progressive decomposition avoids this exponential complexity by exploiting the path exhaustive property of diamonds: once an inner diamond is solved, its conditional probabilities remain valid regardless of how outer diamonds condition on different states. This allows diamond structures to be replaced with supernodes that preserve exact mathematical relationships while eliminating the need for repeated conditioning. The following lemmas establish the mathematical foundation for the decomposition method.

Lemma 7.1. *Invariance of Inner Diamond* Let D_{outer} be a diamond containing an inner diamond D_{inner} where the join node of D_{inner} is an intermediate node in the paths of D_{outer} . Then the conditional probabilities $P(J_{\text{inner}}|F_{\text{inner}})$ and $P(J_{\text{inner}}|\bar{F}_{\text{inner}})$ are invariant to the conditioning state of D_{outer} .

Proof. By the diamond definition, D_{inner} contains all paths between its fork and join nodes. The conditional probabilities depend only on the internal structure of D_{inner} , which remains unchanged regardless of the outer diamond's conditioning state. When D_{outer} conditions on fork state s , the inner diamond's probability becomes:

$$P(\text{signal through } D_{\text{inner}}) = P(J_{\text{inner}}|F_{\text{inner}}) \cdot b(F_{\text{inner}}|s) \\ + P(J_{\text{inner}}|\bar{F}_{\text{inner}}) \cdot (1 - b(F_{\text{inner}}|s)) \quad (18)$$

where $b(F_{\text{inner}}|s)$ is the fork belief given outer conditioning state s , but the conditional terms $P(J_{\text{inner}}|F_{\text{inner}})$ and $P(J_{\text{inner}}|\bar{F}_{\text{inner}})$ remain constant. $\square$

Lemma 7.2 (Supernode Equivalence). *Let D_{inner} be a diamond with fork node F_{inner} and join node J_{inner} . When D_{inner} is replaced with a supernode, the supernode stores the conditional probabilities $P(J_{\text{inner}}|F_{\text{inner}})$ and $P(J_{\text{inner}}|\bar{F}_{\text{inner}})$. For any outer conditioning context with fork belief $b(F_{\text{inner}}|s)$, the supernode's contribution is:*

$$P(\text{supernode contribution}) = P(J_{\text{inner}}|F_{\text{inner}}) \cdot b(F_{\text{inner}}|s) \\ + P(J_{\text{inner}}|\bar{F}_{\text{inner}}) \cdot (1 - b(F_{\text{inner}}|s)) \quad (19)$$

Proof. The conditional probabilities $P(J_{\text{inner}}|F_{\text{inner}})$ and $P(J_{\text{inner}}|\bar{F}_{\text{inner}})$ depend only on D_{inner} 's internal structure and remain invariant across different conditioning contexts. By storing these conditional terms rather than a fixed probability, the supernode can correctly compute its contribution for any fork belief value $b(F_{\text{inner}}|s)$ determined by the outer diamond's conditioning state s . This preserves the exact mathematical relationship that would exist if D_{inner} were processed directly within each conditioning context. $\square$

The diamond-to-supernode decomposition serves two purposes: first, it eliminates the need to re-condition on the same diamond if it appears as a part of a larger diamond's intermediate nodes. Secondly, collapsing the diamond edges always simplifies the network structure for later diamonds further in the topological sorting (Table 1).

To further illustrate the diamond to supernode decomposition process, consider the nested diamond network shown in Figure 5:

The topological processing order guarantees that node 6's belief is updated before any deeper join nodes. When processing node 6, the algorithm discovers diamond D1, which consists of the shared fork ancestor node 1 with divergent paths $1 \rightarrow 2 \rightarrow 5 \rightarrow 6$ and $1 \rightarrow 3 \rightarrow 6$. The diamond is solved by conditioning only on node 1's state. Once solved, the entire diamond subgraph (nodes 2, 3, 5, 6, 13) is replaced with supernode D1. With D1 reduced to a supernode, the algorithm continues with a reduced network structure. The same process is repeated later for D2 and D3 with each decomposition step reducing the computational burden for later diamonds. By the time the algorithm reaches the final sink node 16, only $O(2^1)$ conditioning states (node 1 active/inactive) are needed since supernodes D2 and D3 have absorbed and preserved the dependency introduced from the inner fork nodes 7 and 14.

Table 1: Complexity Comparison for belief updates with vs without progressive decomposition

Type	No Decomposition	With Decomposition
Single Parent (Case 1)	$O(n)$	$O(n)$
Independent Multi Parent (Case 2)	$O(n \times 2^k)$	$O(n \times 2^k)$
Normal diamond (Case 3)	$O(2^c \times d)$	$O(2^c)$
Nested diamonds (Case 4+)	$O(2^{\sum c_i} \times \prod d_i)$	$O(\sum 2^{c_i})$

n = number of nodes, k = max parents per node, c = conditioning nodes per diamond, d = diamond size, c_i = conditioning nodes in diamond i , d_i = size of diamond i

8. Case Studies

8.1. Comparison to dPrPm Approach

Both our Information Propagation Algorithm (IPA) and Tong and Tien’s directed Probability Propagation Method (dPrPm) [21] aim to evaluate network reliability through message propagation. The similarities between these approaches stem from the fundamental acyclic nature of directed acyclic graphs. Both methods:

- Process nodes in a topologically sorted order to ensure dependencies are properly handled,
- Leverage the network’s directional structure to propagate probabilities from source to sink nodes, and
- Aim to reduce computational complexity compared to traditional enumeration methods.

However, the methods diverge in their implementation of how messages between nodes are computed. In dPrPm, when multiple predecessors influence a node, the update rules explicitly rely on approximating a three-node joint distribution for every new edge introduced. These rules are fundamental when the incoming messages are combined in an intermediate structure, where the new link addition affects nodes that have already received some information, requiring a more nuanced treatment of shared dependencies. Although effective, this approach requires storing both marginal and pairwise probabilities throughout the network, which can become memory-intensive in large or dense networks.

IPA takes a fundamentally different view of network dependencies by explicitly modelling them through diamond structures. Rather than tracking pairwise relationships, it identifies and decomposes these structures into statistically equivalent supernodes. As a result, IPA avoids maintaining or updating pairwise dependencies at complex intersection points, since the relevant structure is captured inherently in the decomposition.

To compare the two methods, we evaluate IPA on the same grid network used in [21] (see Figure 6), where all nodes have prior probability 1.0 and all edges have probability 0.9.

Table 2 shows both methods achieve high accuracy compared to the exact solution. dPrPm provides guaranteed bounds with most nodes having no

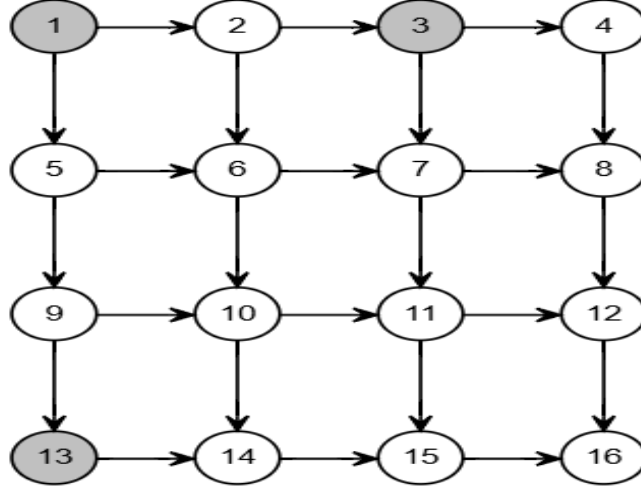


Figure 6: 16-node directed grid network [21]. Source nodes 1, 3, and 13 are coloured in grey.

gap for this network, while IPA computes exact reliability values that match naive path enumeration results. The small errors shown for IPA come from numerical precision errors from floating-point arithmetic.

Table 3 compares the computational performance of both belief propagation approaches. IPA completes the network analysis in 2.378ms compared to dPrPm’s 0.32s, representing a 134-times speedup while requiring only 4.94MB of memory. This performance improvement stems directly from IPA’s diamond decomposition strategy, which eliminates the need to track pairwise dependencies and enables parallel processing within topological layers.

Table 2: Comparison of results from IPA and dPrPm for grid network ($R_l = 0.9$)

Node	Exact	IPA	Abs. Error	Error (%)	dPrPm (Lower)	dPrPm (Upper)	Gap	dPrPm Error (%)
1	1.00000	1.00000	0.00000	0.00000	1.00000	1.00000	0.00000	0.00000
2	0.99000	0.99000	0.00000	0.00000	0.99000	0.99000	0.00000	0.00000
3	1.00000	1.00000	0.00000	0.00000	1.00000	1.00000	0.00000	0.00000
4	0.98015	0.98015	8.88×10^{-16}	9.06×10^{-12}	0.98015	0.98015	0.00000	0.00000
5	0.90000	0.90000	0.00000	0.00000	0.90000	0.90000	0.00000	0.00000
6	0.99510	0.99510	9.99×10^{-16}	1.00×10^{-11}	0.99510	0.99510	0.00000	0.00000
7	0.98956	0.98956	4.44×10^{-16}	4.49×10^{-12}	0.98956	0.98956	0.00000	0.00000
8	0.89060	0.89060	5.55×10^{-16}	6.23×10^{-12}	0.89060	0.89060	0.00000	0.00000
9	0.98100	0.98100	0.00000	0.00000	0.98100	0.98100	0.00000	0.00000
10	0.88290	0.88290	1.11×10^{-16}	1.26×10^{-12}	0.88290	0.88290	0.00000	0.00000
11	0.97734	0.97734	2.44×10^{-15}	2.50×10^{-11}	0.97734	0.97734	0.00000	0.00000
12	0.97457	0.97457	2.66×10^{-15}	2.73×10^{-11}	0.97457	0.97457	0.00000	0.00000
13	1.00000	1.00000	0.00000	0.00000	1.00000	1.00000	0.00000	0.00000
14	0.97946	0.97946	2.22×10^{-16}	2.27×10^{-12}	0.97946	0.97946	0.00000	0.00000
15	0.98498	0.98498	1.22×10^{-15}	1.24×10^{-11}	0.98498	0.98498	0.00000	0.00000
16	0.98539	0.98539	1.49×10^{-11}	1.51×10^{-7}	0.98539	0.98539	0.00000	0.00000

Table 3: IPA vs dPrPm performance comparison

Method	Runtime	Memory
IPA Belief Propagation	2.378 ms	4.94 MB
dPrPm Belief Propagation ^a	0.32 s	N/A ^a

^aResults from [21] - memory profiling data not available

8.2. Drone Medical Delivery Network

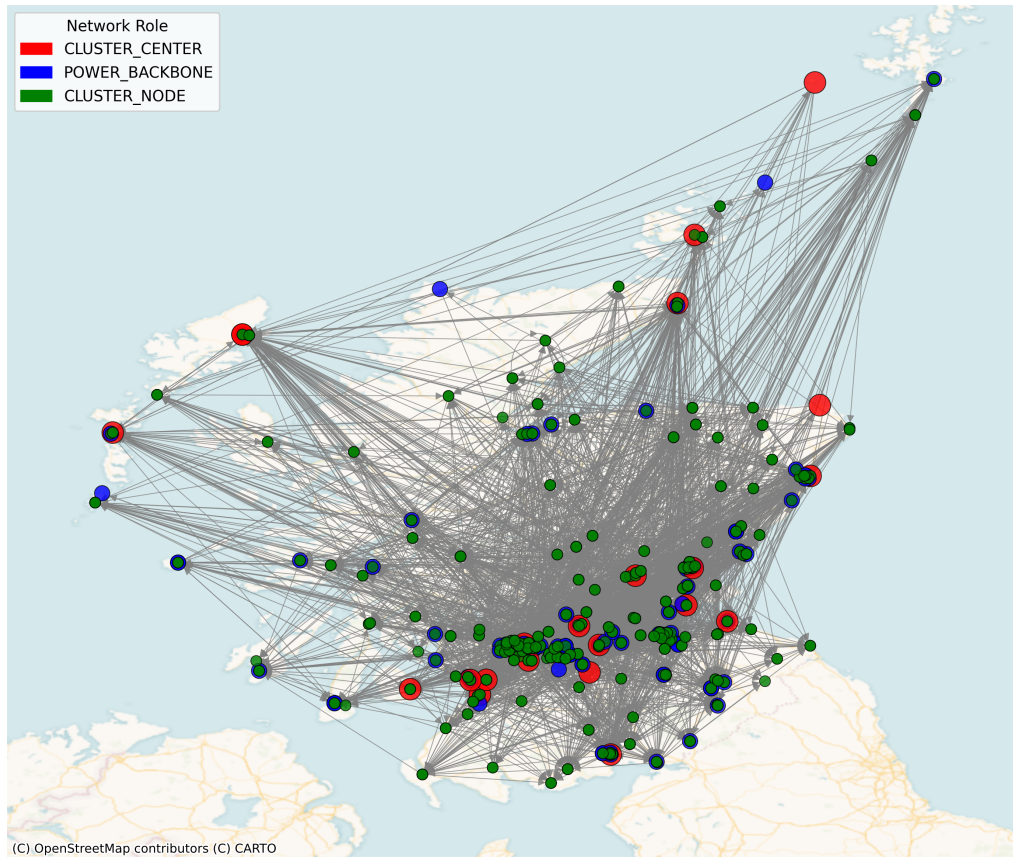


Figure 7: Delivery Network Across Medical Facilities in Scotland

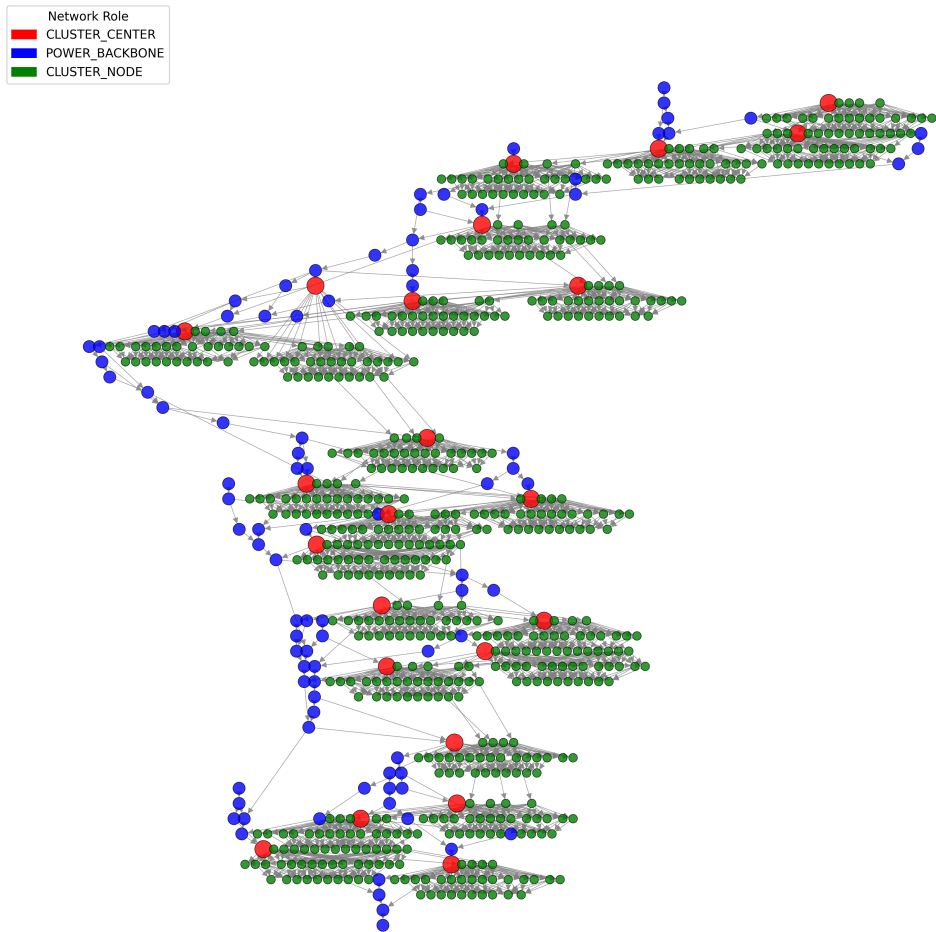


Figure 8: Medical Delivery Network Hierarchical Structure

9. Conclusion

In complex process systems, evaluating signal propagation at all network nodes is invaluable for system design and maintenance planning. Most exact path enumeration methods focus only on sink nodes due to the exponential cost of calculating and combining all distinct paths from every source to every node.

This paper presents the Information Propagation Algorithm (IPA), a message passing algorithm to evaluate signal reception reliability in acyclic process systems. The proposed method leverages conditional probability theory in DAGs to resolve dependencies in diamond subgraphs. By resolving and decomposing discovered diamond subgraphs into supernodes, the network structure is progressively simplified. This reduces the complexity from $O(2^{\sum c_i} \times \prod d_i)$ without decomposition to $O(\sum 2^{c_i})$ with decomposition for subsequent belief updates on the same diamond structure.

Two case studies were presented to validate IPA’s effectiveness and accuracy. First, we compared our approach to dPrPm, another message passing algorithm for signal propagation in DAGs. On the same grid network, IPA computed the same result as exact path enumeration while being 134-times faster than dPrPm. The second case study applied IPA to a 782-node medical delivery network with 2,305 unique decomposed supernodes, confirming accuracy with an absolute error of 0.0016 compared to Monte Carlo simulation, while completing 36

Although exact, IPA struggles in pathologically dense networks where diamond structures span multiple topological layers. The diamond conditioning process effectively becomes full enumeration since decomposition then cannot meaningfully simplify the DAG structure.

Future research directions include developing optimization algorithms that pre-decompose the full network before belief propagation. This could be particularly useful in dense networks where only a subset of node reliabilities are required as IPA can then be applied to the decomposed network that would otherwise be beyond IPA’s practical limits. Additionally, further parallelization opportunities exist beyond topological layer processing, particularly during diamond decomposition when nested diamonds are not interconnected.

References

- [1] H. George-Williams, T. V. Santhosh, E. Patelli, Simulation Methods for the Analysis of Complex Systems, 2021. doi:10.1007/978-3-030-83640-5_7.
URL https://link.springer.com/chapter/10.1007/978-3-030-83640-5_7
- [2] R. Billinton, R. N. Allan, Network modelling and evaluation of simple systems, in: R. Billinton, R. N. Allan (Eds.), Reliability Evaluation of Engineering Systems: Concepts and Techniques, Springer US, Boston, MA, 1992, pp. 81–100. doi:10.1007/978-1-4899-0685-4_4.
URL https://doi.org/10.1007/978-1-4899-0685-4_4
- [3] Rausand, M. (2014), Reliability Quantification, in: Reliability of Safety-Critical Systems, John Wiley & Sons, Ltd, 2014, pp. 91–164, section: 5 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118776353.ch5>. doi:<https://doi.org/10.1002/9781118776353.ch5>.
URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781118776353.ch5>
- [4] C. Ebeling, An Introduction to Reliability and Maintainability Engineering: Third Edition, Waveland Press, 2019.
URL <https://books.google.co.uk/books?id=rh2WDwAAQBAJ>
- [5] S. Distefano, A. Puliafito, Dynamic Reliability Block Diagrams VS Dynamic Fault Trees, 2007 Proceedings - Annual Reliability and Maintainability Symposium (2007) 71–76Conference Name: 2007 Proceedings Annual Reliability and Maintainability Symposium ISBN: 9780780397668 9780780397675 Place: Orlando, FL, USA Publisher: IEEE. doi:10.1109/RAMS.2007.328095.
URL <http://ieeexplore.ieee.org/document/4126327/>
- [6] L. Xing, S. V. Amari, Fault Tree Analysis, in: K. B. Misra (Ed.), Handbook of Performability Engineering, Springer London, London, 2008, pp. 595–620. doi:10.1007/978-1-84800-131-2_38.
URL https://doi.org/10.1007/978-1-84800-131-2_38

- [7] M. Čepin, Event Tree Analysis, in: *Assessment of Power System Reliability: Methods and Applications*, Springer London, London, 2011, pp. 89–99. doi:10.1007/978-0-85729-688-7_6.
URL https://doi.org/10.1007/978-0-85729-688-7_6
- [8] D. J. Sherwin, A. Bossche, Fault trees, event trees and success trees, in: *The Reliability, Availability and Productiveness of Systems*, Springer Netherlands, Dordrecht, 1993, pp. 133–161. doi:10.1007/978-94-011-1582-7_9.
URL https://doi.org/10.1007/978-94-011-1582-7_9
- [9] A. Rauzy, New algorithms for fault trees analysis, *Reliability Engineering & System Safety* 40 (3) (1993) 203–211. doi:[https://doi.org/10.1016/0951-8320\(93\)90060-C](https://doi.org/10.1016/0951-8320(93)90060-C).
URL <https://www.sciencedirect.com/science/article/pii/095183209390060C>
- [10] Bryant, Graph-Based Algorithms for Boolean Function Manipulation, *IEEE Transactions on Computers* C-35 (8) (1986) 677–691. doi:10.1109/TC.1986.1676819.
- [11] K. S. Brace, R. L. Rudell, R. E. Bryant, Efficient implementation of a BDD package, *Conference proceedings on 27th ACM/IEEE design automation conference - DAC '90* (1990) 40–45Conference Name: Conference proceedings ISBN: 9780897913638 Place: Orlando, Florida, United States Publisher: ACM Press. doi:10.1145/123186.123222.
URL <http://portal.acm.org/citation.cfm?doid=123186.123222>
- [12] R. E. Bryant, C. Meinel, *Ordered Binary Decision Diagrams*. doi:10.1007/978-1-4615-0817-5_11.
URL https://link.springer.com/chapter/10.1007/978-1-4615-0817-5_11
- [13] A. Mosleh, *Estimation of Parameters of Common Cause Failure Models*. doi:10.1007/978-94-017-0629-2_12.
URL https://link.springer.com/chapter/10.1007/978-94-017-0629-2_12
- [14] G. Fishman, A Comparison of Four Monte Carlo Methods for Estimating the Probability of s-t Connectedness, *IEEE Transactions on*

- Reliability 35 (2) (1986) 145–155. doi:10.1109/TR.1986.4335388.
 URL <https://www.scopus.com/inward/record.uri?eid=2-s2.0-0022736325&doi=10.1109%2fTR.1986.4335388&partnerID=40&md5=80d0fbdb617df7c3757577fbd528b559>
- [15] K.-P. Hui, Monte Carlo Network Reliability Ranking Estimation, IEEE Transactions on Reliability 56 (1) (2007) 50–57. doi:10.1109/TR.2006.890898.
- [16] S.-K. Au, J. L. Beck, Estimation of small failure probabilities in high dimensions by subset simulation, Probabilistic Engineering Mechanics 16 (4) (2001) 263–277. doi:[https://doi.org/10.1016/S0266-8920\(01\)00019-4](https://doi.org/10.1016/S0266-8920(01)00019-4).
 URL <https://www.sciencedirect.com/science/article/pii/S0266892001000194>
- [17] T. B. Kumar, O. C. Sekhar, M. Ramamoorthy, Composite power system reliability evaluation using modified minimal cut set approach, Alexandria Engineering Journal 57 (4) (2018) 2521–2528. doi:<https://doi.org/10.1016/j.aej.2017.09.008>.
 URL <https://www.sciencedirect.com/science/article/pii/S1110016817302934>
- [18] H. Li, Q. Zhao, A cut/tie set method for reliability evaluation of control systems, in: Proceedings of the 2005, American Control Conference, 2005., 2005, pp. 1048–1053 vol. 2, iSSN: 2378-5861. doi:10.1109/ACC.2005.1470099.
- [19] W. Liu, J. Li, An improved cut-based recursive decomposition algorithm for reliability analysis of networks, Earthquake Engineering and Engineering Vibration 11 (1) (2012) 1–10. doi:10.1007/s11803-012-0093-5.
 URL <http://link.springer.com/10.1007/s11803-012-0093-5>
- [20] Y. Kim, W.-H. Kang, Network reliability analysis of complex systems using a non-simulation-based method, Reliability Engineering & System Safety 110 (2013) 80–88. doi:10.1016/j.ress.2012.09.012.
 URL <https://www.sciencedirect.com/science/article/pii/S0951832012001925>

- [21] Y. Tong, I. Tien, Probability Propagation Method for Reliability Assessment of Acyclic Directed Networks, ASCE-ASME Journal of Risk and Uncertainty in Engineering Systems, Part A: Civil Engineering 5 (3) (2019) 04019011, _eprint: <https://ascelibrary.org/doi/pdf/10.1061/AJRUA6.0001017>, doi:10.1061/AJRUA6.0001017.
URL <https://ascelibrary.org/doi/abs/10.1061/AJRUA6.0001017>
- [22] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann series in representation and reasoning, Elsevier Science, 1988.
URL <https://books.google.co.uk/books?id=AvNID7LyMusC>
- [23] J. S. Yedidia, W. T. Freeman, Y. Weiss, Understanding belief propagation and its generalizations, in: Exploring Artificial Intelligence in the New Millennium, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2003, pp. 239–269.
- [24] K. Murphy, Y. Weiss, M. Jordan, Loopy Belief Propagation for Approximate Inference: An Empirical Study, Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (Jan. 2013).